

Program partitioning and scheduling

Grain size or granularity: (defines level of parallelism)

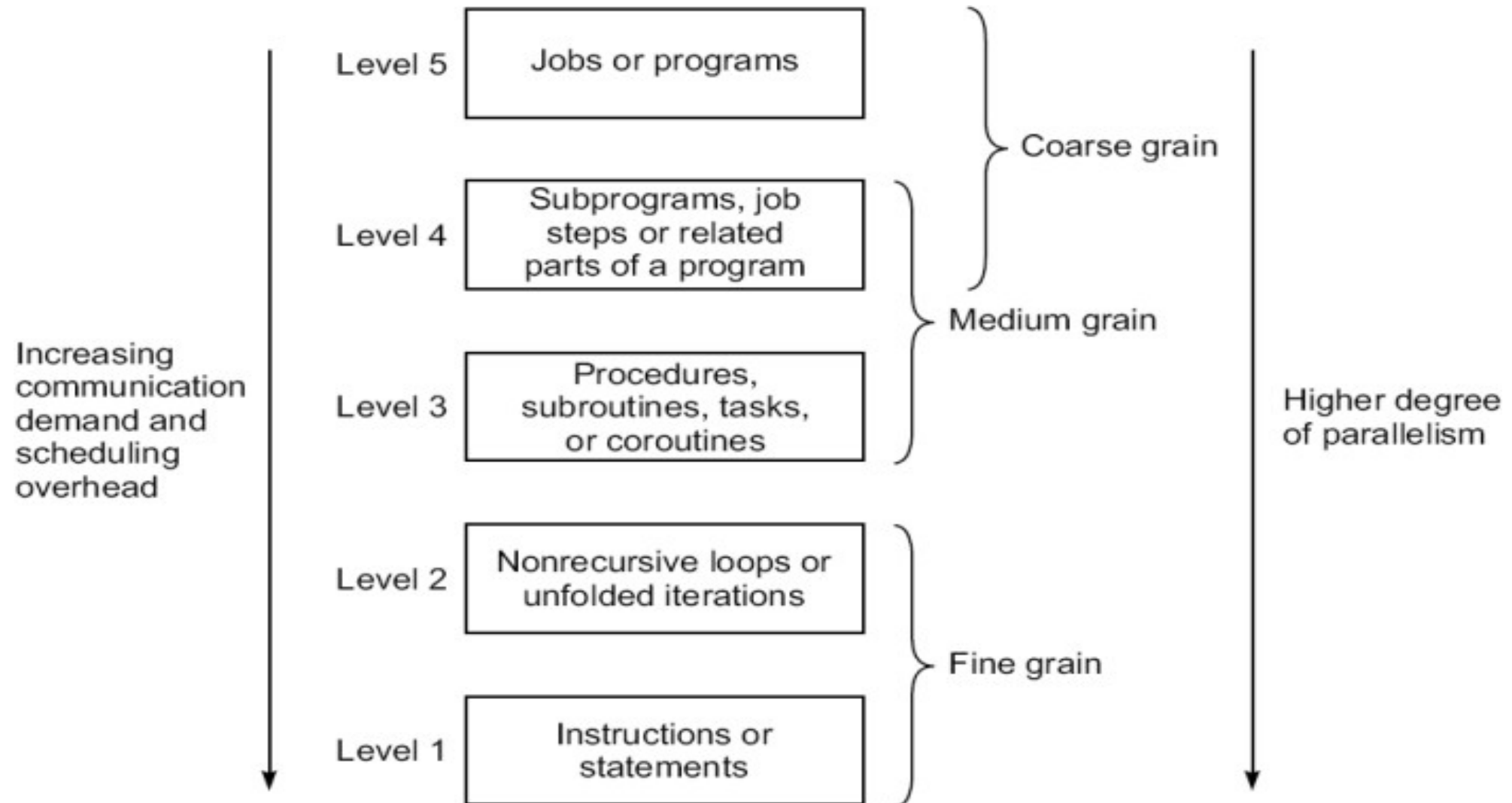
- The size of the parts or pieces of a program that can be considered for parallel execution can vary. The sizes are roughly classified using the term “granule size,” or simply “granularity.” for example, the number of instructions in a program part.
- Grain sizes are usually described as fine, medium or coarse, depending on the level of parallelism involved.

Latency

- Latency is the time required for communication between different subsystems in a computer.
- **Memory latency**, for example, is the time required by a processor to access memory.
- **Synchronization latency** is the time required for two processes to synchronize their execution.

Computational granularity and communication latency are closely related

Processing levels of a program (lower the level, finer the granularity of the software process)



The execution of a program may involve the combination of these levels. But the actual combination depends on the application, formulation, algorithm, language, program, compilation support and hardware characteristics.

Levels of Parallelism:

Instruction Level Parallelism

The fine-grained, or smallest granularity level typically involves less than 20 instructions per grain. The number of candidates for parallel execution varies from 2 to thousands.

Advantages:

There are usually many candidates for parallel execution

Compilers can usually do a reasonable job of finding this parallelism

Loop level parallelism (level 2)

- This corresponds to the iterative loop operations. A typical loop contains less than 500 instructions.
- If a loop operation is independent between iterations, it can be handled by a pipeline, or by a SIMD machine.
- Loop-level parallelism is the most optimized program construct to execute on a parallel or vector computer.

Procedure-level Parallelism

- Medium-sized grain; usually less than 2000 instructions.
- Detection of parallelism is more difficult than with smaller grains.
- interprocedural dependence analysis is difficult and history-sensitive.
- SPMD (single procedure multiple data) is a special case in this level.
- Multitasking belongs to this level.

Subprogram-level Parallelism

- grain typically has thousands of instructions.
- medium- or coarse-grain level.
- Job steps can overlap across different jobs.
- Multiprogramming conducted at this level. No compilers available to exploit medium- or coarse-grain parallelism at present.
- Parallelism at this level is exploited by algorithm designers or programmers and not by compilers.

Job or Program-Level Parallelism

- Corresponds to execution of essentially independent jobs or programs on a parallel computer.
- This is practical for a machine with a small number of powerful processors, but impractical for a machine with a large number of simple processors (since each processor would take too long to process a single job).
- The grain size can be as high as millions of instructions in a single program.
- For supercomputers with a small number of very powerful processors, this coarse-grain parallelism is practical.

Grain packing and scheduling

- Two fundamental questions arise in parallel programming:
 1. How can we partition a program into parallel branches or granules?
 2. What should be the optimal size of concurrent grains in a computation?

There is an obvious tradeoff between parallelism and scheduling/synchronization overhead.

One approach to the problem is called “grain packing.”

Example - Program graph and grain packing

VAR a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q

BEGIN

1) a := 1

2) b := 2

3) c := 3

4) d := 4

5) e := 5

6) f := 6

7) g := a x b

8) h := c x d

9) i := d x e

10) j := e x f

11) k := d x f

12) l := j x k

13) m := 4 x 1

14) n := 3 x m

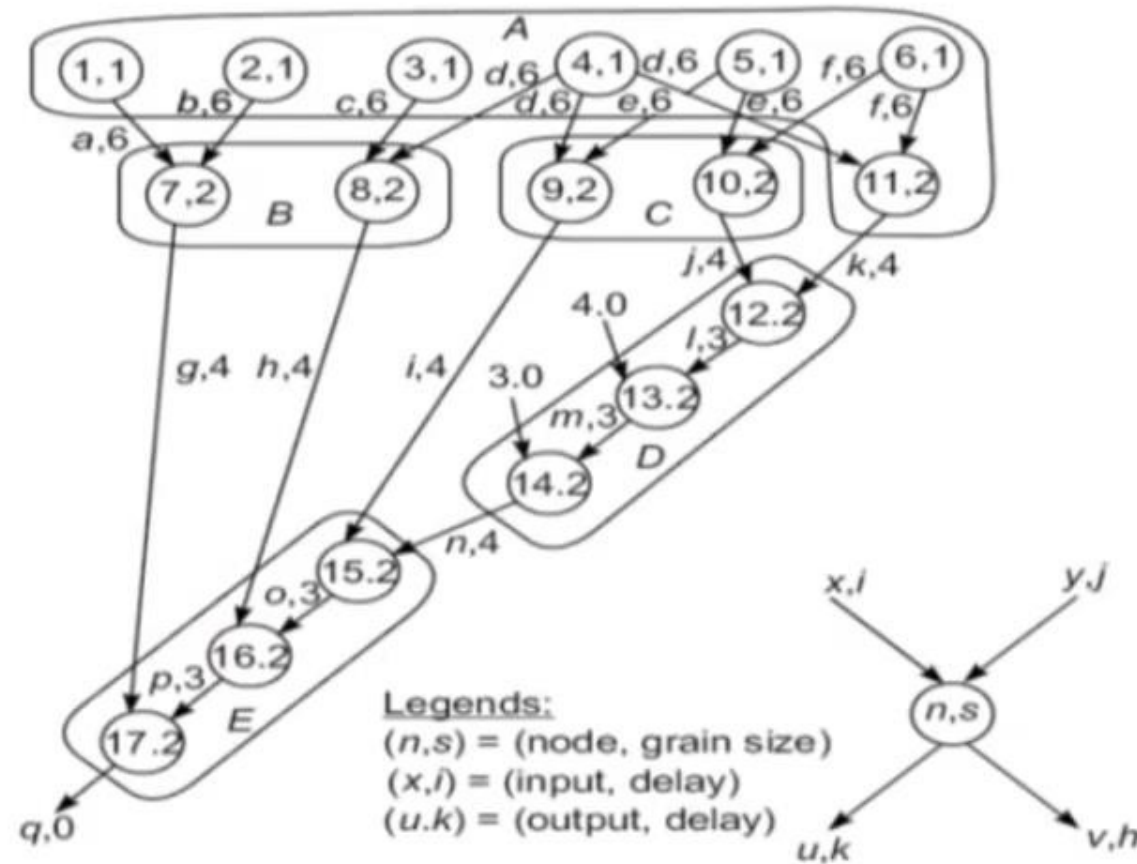
15) o := n x i

16) p := o x h

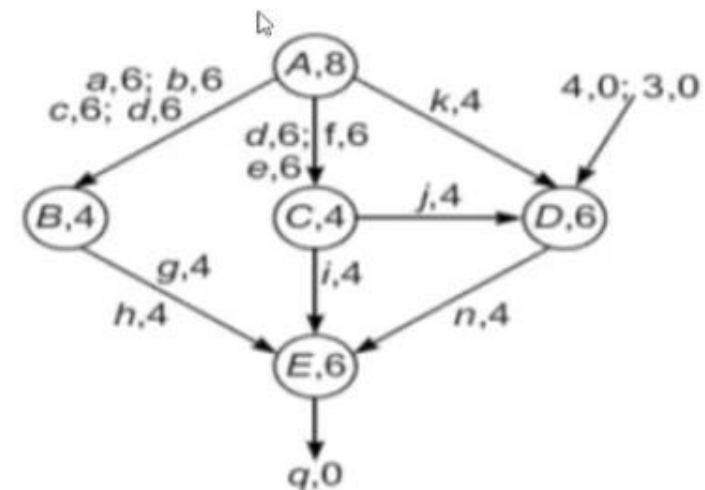
17) q := p x q

END

Example - Program graph and grain packing contd.



(a) Fine-grain program graph before packing



(b) Coarse-grain program graph after packing

Fig. 2.6 A program graph before and after grain packing in Example 2.4 (Modified from Kruatrachue and Lewis, *IEEE Software*, Jan. 1988)

- Each node in the program graph shows the computational unit in the program (total 17 computational units).
- The grain size is measured by the number of basic machine cycles including processor cycle and memory cycle needed to execute all operations within the node.
- Each node is denoted by a pair (n, s) where n is node name and s is the grain size.
- So in above fig. fine grain nodes have smaller grain size and coarse-grain nodes have larger grain size.
- The edge label (v, d) between two nodes specifies the output variable v from the source node or input variable to the destination node, and the communication delay between them is denoted by d .
- This delay includes all the path delay and memory latency.
- The fine grain graph has 17 nodes and coarse-grain graph has 5 nodes which are obtained by combining multiple fine grain nodes.
- Nodes 1,2,...,6 are memory reference (data fetch) operations and each takes one cycle to address and six cycles to fetch from memory.
- All remaining nodes from 7 to 17 are CPU operations, each require 2 cycles to complete.
- The node $(A, 8)$ is obtained by combining nodes $(1,1)$, $(4,1)$, $(5,1)$, $(6,1)$, and $(11,2)$.